



دانشکده مهندسی کامپیوتر
دانشگاه صنعتی شریف

آزمون پذیری
فاز دوم پروژه

نام و نام خانوادگی:

مهران تقوی

استاد : دکتر حسابی

زمستان 1403

صورت مسئله:

استفاده از زبان‌های برنامه نویسی یک شبیه‌ساز طراحی و پیاده‌سازی نمایید که فایل‌های استاندارد ISCAS را بخواند و این شبیه‌ساز بایستی قادر باشد که پارامترهای SCOAP تمام نت‌ها و خروجی‌ها را تحویل دهد. گام دوم این پروژه پیاده‌سازی الگوریتم PODEM است برای تولید بردار تست.

پیاده‌سازی:

برای پیاده‌سازی این بخش باید پکیج‌های مورد نیاز و ضروری را اضافه کنیم.

```
1 from enum import Enum
2 from itertools import product
3 import math
4
```

مقادیر پذیرفته شده را برای سیگنال‌هایمان و اشکال‌ها تعریف می‌کنیم:

```
class FaultType(Enum):
    SA0 = "sa-0"
    SA1 = "sa-1"

class ValueType(Enum):
    One = "1"
    Zero = "0"
    Unknown = "U"
    HighZ = "Z"
    # For fault
    One_Zero = "D"
    Zero_One = "~D"
```

حال یک کلاس به نام Testability استفاده کرده‌ایم، که در این کلاس به این صورت عمل می‌کنیم ابتدا اون فایل isc را می‌خوانیم، که چون مدار ما به صورت ترکیبی می‌باشد برای ورودی‌ها قابلیت کنترل 0 و 1 را به ترتیب برابر با 1 قرار می‌دهیم و از هر گیتی که رد میشه مقدار خروجی آن را محاسبه می‌شود و 1 واحد به آن اضافه می‌شود. و برای fanout‌ها برابر مقدار ساقه هاش قرار می‌دهیم و به همین صورت عمل می‌کنیم تا به آخر برسیم و در ادامه برای هر گیت تابع آن را قرار می‌دهیم. و برای محاسبه قابلیت مشاهده هر خط از مدار از خروجی شروع می‌کنیم برای خروجی مقدار آن را برابر با 0 قرار می‌دهیم و از هر گیتی که رد می‌شویم با توجه به نوع گیت آن مقدار آن را حساب می‌کنیم و 1 واحد به آن اضافه می‌کنیم و برای شاخه‌های fanout‌ها برابر با مقدار ورودی آن گیت مربوط قرار می‌دهیم و دلیل این کار هم این است که از آخر به سمت عقب می‌ایم و برای ساقه

ها طبق چیزی که در درس می باشد برابر با مینیم fanout ها قرار می دهیم و این روند ادامه پیدا می کند تا به ورودی ها برسیم.

و این روند چیزی که گفته شده است با استفاده از تابع ها و کلاس به زبان پایتون پیاده کرده ایم.

```
1 class Testability:
2     operations = {}
3     CC_CO = {}
4
5     def addOperation(self, name: str, operation):
6         self.name = name
7         self.operations[name] = operation
8
9     def getValue(self, operation_name : str, inputs : list[ValueType]):
10        operation = self.operations.get(operation_name)
11        return operation(inputs)
12
13    def calculateControllability(self, netList: list):
14        for net in netList:
15            thisNetList = netList[net]
16            gateType = thisNetList[2]
17            if gateType == "inpt":
18                self.CC_CO[net] = {"CC": [1, 1]}
19            elif gateType == "from":
20                self.CC_CO[net] = {"CC": [self.CC_CO[thisNetList[3].replace("gat", "")]["CC"][0],
21                self.CC_CO[thisNetList[3].replace("gat", "")]["CC"][1]]}
22            else:
23                inputNets = self.getInputNets(thisNetList=thisNetList)
24                if gateType in ["not", "buff", "buf"]:
25                    inputControllabilities = self.getInputsControllabilities(inputNets=inputNets)[0]
26                    else:
27                        inputControllabilities = self.getInputsControllabilities(inputNets=inputNets)
28                    operation = self.operations.get(f"{gateType}Ctrl")
29                    self.CC_CO[net] = {"CC": operation(inputControllabilities)}
30
31    def getInputNets(self, thisNetList: list[dict]):
32        if thisNetList[2] in ("not", "buf", "buff"):
33            return [thisNetList[len(thisNetList) - 2]]
34        else:
35            inputStartIndex = len(thisNetList) - 1 - int(thisNetList[4])
36            inputEndIndex = len(thisNetList) - 1
37            return thisNetList[inputStartIndex: inputEndIndex]
38
39    def getInputsControllabilities(self, inputNets: list) -> list:
40        return [self.CC_CO[input]["CC"] for input in inputNets]
41
42    def reverseNetList(self, netList: list) -> list:
43        return dict(reversed(netList.items()))
44
45    def calculateObservability(self, netList: list):
46        reversedNetList = self.reverseNetList(netList)
47        for net in self.CC_CO:
48            self.CC_CO[net]["CO"] = math.inf
49        for net, thisNetList in reversedNetList.items():
50
51            if thisNetList[3] == "0":
52                self.CC_CO[net]["CO"] = 0
```

حال یک تابع به نام readISC استفاده کرده ایم:

تابع readISC که کد آن در زیر نیز آورده شده است، فایل isc را بصورت خط به خط خوانده و در یک لیست ذخیره می کند و همچنین خطوطی را که با * شروع شده اند (خطوط comment) را اصلاً نمی خواند. به عبارتی ما یک دیکشنری به نام netList داریم که هر عنصر این دیکشنری، یک لیست حاوی یک خط از فایل isc است.

```
def readISC(self, file) -> dict:
    netlist = {}
    isInputsLine = False
    lastNet = None
    for line in file.readlines():
        if line.startswith('*'):
            continue
        tmp = line.strip().split()

        if isInputsLine:
            tmp = netlist[lastNet] + tmp
            tmp.append(ValueType.Unknown)
            isInputsLine = False

        elif tmp[2] == "inpt" or tmp[2] == "from":
            tmp.append(ValueType.Unknown)

        else:
            isInputsLine = True
            lastNet = tmp[0]

        netlist[tmp[0]] = tmp

    return netlist
```

در اینجا به معرفی تابع ها برای محاسبه قابلیت کنترل و مشاهده گیت ها می پردازیم.

به طور مثال برای گیت And قابلیت کنترل مقدار 0 در خروجی برابر است با مینیمم قابلیت کنترل 0 در ورودی ها به علاوه ی یک و قابلیت کنترل مقدار 1 در خروجی برابر است با جمع قابلیت کنترل مقدار یک در تمامی ورودی ها. و قابلیت مشاهده در ورودی ها برابر است با قابلیت مشاهده در خروجی به علاوه ی قابلیت کنترل مقدار 1 در تمامی ورودی های دیگر. برای گیت Xor این داستان متفاوت میباشد برای بدست آوردن قابلیت کنترل مقدار یک در خروجی حالت های مختلفی که تعداد یک ها در ورودی فرد میشود بدست میاوریم و سپس مینیمم آن ها را به علاوه یک به عنوان قابلیت کنترل یک در خروجی قرار می دهیم و برای قابلیت کنترل صفر در

خروجی مینیمم حالت هایی را در نظر میگیریم که تعداد یک ها فرد است. و قابلیت مشاهده XOR برای هر ورودی مینیمم قابلیت کنترل آن (0 یا 1) را انتخاب کرده و جمع آن ها را حساب میکنیم و به علاوه 1 میکنیم.

```
def calculateObservability(self, netList: list):
    reversedNetList = self.reverseNetList(netList)
    for net in self.CC_CO:
        self.CC_CO[net]["CO"] = math.inf
    for net, thisNetList in reversedNetList.items():

        if thisNetList[3] == "0":
            self.CC_CO[net]["CO"] = 0

        if thisNetList[2] == "from":
            fanoutStemName = thisNetList[3].replace("gat", "")
            netListKeys = list(netList.keys())
            fanoutCount = int(netList[fanoutStemName][3])
            fanoutBranchRangeStartIndex = netListKeys.index(fanoutStemName) + 1
            fanoutBranchRangeEndIndex = fanoutBranchRangeStartIndex + fanoutCount
            self.CC_CO[fanoutStemName]["CO"] = min([self.CC_CO[branch]["CO"]
                                                    for branch in
                                                    netListKeys[fanoutBranchRangeStartIndex:
                                                                    fanoutBranchRangeEndIndex]]) + 1
            continue

        inputStartIndex = len(thisNetList) - 1 - int(thisNetList[4])
        inputEndIndex = len(thisNetList) - 1
        inputs = thisNetList[inputStartIndex: inputEndIndex]
        gateType = thisNetList[2]
        self.assignGateInputsObservabilities(gateType, inputs, net)
```

```
def assignGateInputsObservabilities(self, gateType: str, inputs: list, net: str):
    if len(inputs) == 1:
        self.CC_CO[inputs[0]]["CO"] = self.CC_CO[net]["CO"] + 1
        return

    for i in inputs:
        self.CC_CO[i]["CO"] = self.CC_CO[net]["CO"] + 1
        for j in inputs:
            if i == j:
                continue
            if gateType in ["nand", "and"]:
                self.CC_CO[i]["CO"] += self.CC_CO[j]["CC"][1]
            elif gateType in ["or", "nor"]:
                self.CC_CO[i]["CO"] += self.CC_CO[j]["CC"][0]
            elif gateType in ["xor", "xnor"]:
                self.CC_CO[i]["CO"] += min(self.CC_CO[j]["CC"])
```

```

def andGateControllability(inputsControllabilities: list[list[int]]) -> list[int]:
    return [min([inputControllability[0] for inputControllability in inputsControllabilities]) + 1,
            sum([inputControllability[1] for inputControllability in inputsControllabilities]) + 1]

obj.addOperation("andCtrl", andGateControllability)

def orGateControllability(inputsControllabilities: list[list[int]]) -> list[int]:
    return [sum([inputControllability[0] for inputControllability in inputsControllabilities]) + 1,
            min([inputControllability[1] for inputControllability in inputsControllabilities]) + 1]

obj.addOperation("orCtrl", orGateControllability)

def nandGateControllability(inputsControllabilities: list[list[int]]) -> list[int]:
    return [sum([inputControllability[1] for inputControllability in inputsControllabilities]) + 1,
            min([inputControllability[0] for inputControllability in inputsControllabilities]) + 1]

obj.addOperation("nandCtrl", nandGateControllability)

def norGateControllability(inputsControllabilities: list[list[int]]) -> list[int]:
    return [min([inputControllability[1] for inputControllability in inputsControllabilities]) + 1,
            sum([inputControllability[0] for inputControllability in inputsControllabilities]) + 1]

```

```

def xorGateControllability(inputs_controllabilities: list[list[int]]) -> list[int]:
    #calculate number of inputs
    inputs_num = len(inputs_controllabilities)
    '''possible_state1/0_controllabilities lists the different controllabilities
    for all different combinations of input bits'''
    possible_state1_controllabilities = []
    possible_state0_controllabilities = []
    #state is every possible combination of 1's and 0's in the input
    states = list(product([0, 1], repeat=inputs_num))
    #states1/0 is every possible state in which there are an odd number of 1's/0's
    states1 = [state for state in states if sum(state) % 2 == 1]
    states0 = [state for state in states if sum(state) % 2 == 0]

    for state in states1:
        controllability = 0
        for i in range(inputs_num):
            #calculates the controllability for a specific state is states1
            controllability += inputs_controllabilities[i][state[i]]

        possible_state1_controllabilities.append(controllability)

    for state in states0:
        controllability = 0
        for i in range(inputs_num):
            #calculates the controllability for a specific state is states0
            controllability += inputs_controllabilities[i][state[i]]

        possible_state0_controllabilities.append(controllability)

    #takes the minimum as xor_controllability
    xor_controllability0 = min(possible_state0_controllabilities)
    xor_controllability1 = min(possible_state1_controllabilities)

    return [xor_controllability0 + 1, xor_controllability1 + 1]

obj.addOperation("xorCtrl", xorGateControllability)

```

برای نمونه چند تابع برای محاسبه قابلیت کنترل و مشاهده برخی گیت ها در بالا آورده شده است.
و بقیه توابع به همین صورت انجام می شود. که در کد اصلی هست.

حالا قابلیت کنترل و مشاهده را به ترتیب برای چندتا فایل تست می کنیم.

فایل c1.isc:

خروجی فایل c5.isc:

net	CC0	CC1	C0

1	1	1	4
2	1	1	4
3	3	2	2
4	1	1	3
5	2	4	0

خروجی فایل c17.isc:

net	CC0	CC1	C0

1	1	1	5
2	1	1	7
3	1	1	6
8	1	1	5
9	1	1	8
6	1	1	8
7	1	1	6
10	3	2	3
11	3	2	6
14	3	2	6
15	3	2	5
16	4	2	4
20	4	2	3
21	4	2	3
19	4	2	3
22	5	4	0
23	5	5	0

بخشی از خروجی فایل c432.isc:

273	271	16	3	27	366	364	47	53	21
274	272	16	3	80	367	365	47	53	18
275	273	16	3	28	368	366	47	53	27
276	274	16	3	27	369	367	47	53	24
277	275	16	3	80	370	368	47	53	28
278	276	16	3	28	371	369	47	53	27
279	277	16	3	27	372	370	47	53	0
280	278	16	3	80	373	371	55	2	17
281	279	16	3	28	374	372	55	2	16
282	280	16	3	27	375	373	55	2	16
283	281	16	3	80	376	374	55	2	19
284	282	16	3	28	377	375	55	2	16
285	283	16	3	27	378	376	55	2	25
286	284	16	3	80	379	377	55	2	22
287	285	16	3	28	380	378	55	2	26
288	286	16	3	27	381	379	55	2	25
289	287	16	3	80	382	380	8	2	11
290	288	16	3	78	383	381	8	2	10
291	289	16	3	78	384	382	8	2	19
292	290	16	3	78	385	383	8	2	9
293	291	16	3	78	386	384	8	2	11
294	292	16	3	78	387	385	8	2	13
295	293	16	3	78	388	386	8	2	10
296	294	16	3	78	389	387	8	2	19
297	295	16	3	78	390	388	8	2	18
298	296	17	28	2	391	389	8	2	24
299	297	17	28	67	392	390	8	2	9
300	298	17	28	20	393	391	8	2	25
301	299	17	28	1	394	392	8	2	11
302	300	4	17	77	395	393	8	2	13
303	301	4	17	77	396	394	8	2	19

بخشی از خروجی فایل c6288.isc:

						net	CC0	CC1	C0
97	1	1	752	1					
98	1	1	748	2					
99	1	1	744	3	1	1	1	3	
100	1	1	740	4	2	1	1	2	
101	1	1	736	5	3	1	1	19	
102	1	1	727	6	4	1	1	29	
103	1	1	313	7	5	1	1	39	
104	1	1	312	8	6	1	1	45	
105	1	1	403	9	7	1	1	51	
106	1	1	485	10	8	1	1	57	
107	1	1	567	11	9	1	1	63	
108	1	1	645	12	10	1	1	69	
109	1	1	723	13	11	1	1	75	
110	1	1	768	14	12	1	1	81	
111	1	1	764	15	13	1	1	87	
112	1	1	760	16	14	1	1	93	
113	1	1	756	17	15	1	1	99	
114	1	1	752	18	16	1	1	105	
115	1	1	748	19	17	1	1	111	
116	1	1	744	20	18	1	1	23	
117	1	1	740	21	19	1	1	22	
118	1	1	736	22	20	1	1	53	
119	1	1	727	23	21	1	1	75	
120	1	1	407	24	22	1	1	97	
121	1	1	406	25	23	1	1	115	
122	1	1	509	26	24	1	1	133	
123	1	1	603	27	25	1	1	151	
124	1	1	697	28	26	1	1	169	
125	1	1	776	29	27	1	1	187	
126	1	1	772	30	28	1	1	205	
				31	29	1	1	223	

شروع بخش دوم از پروژه الگوریتم PODEM:

در این پروژه از الگوریتم PODEM برای تولید بردارهای تست استفاده شده است. به این صورت که لیستی از خطاهای stuck at به شبیه ساز داده می شود و شبیه ساز نیز بر اساس مقادیر مشخص شده اقدام به تولید بردار تست مناسب می کند. این شبیه ساز فایل fault_file را به عنوان ورودی دریافت کرده و فایل

test_vectors را به عنوان خروجی تحویل میدهد. فایل به این صورت است که شبیه سازی که نوشتیم فایل .isc را میخواند فایل faultFile را تولید می کند این شبیه ساز به ازای هر یک از خطاهای مشخص شده در faultFile بردار تست را تولید کرده و در فایل خروجی ذخیره می کند.

در ابتدا به این صورت عمل میکنیم قابلیت کنترل را برای تمام خطوط مدار را بدست میاریم که اینو در مرحله اول پروژه فاز دوم بدست آوردیم.

تابع PODEM به صورت ماشین حالت پیاده سازی شده است. که چند حالت به نام های زیر دارد.

Start, XPathCheck, Objective, BackTrack, BackTrace, Implication, CheckDetected, Finish_Success, Finish_Untestable

PODEM Class کلاسی است که در آن تمام توابع مورد نیاز برای الگوریتم PODEM را پیاده سازی کرده ایم. این کلاس شامل توابع زیر است.

تابع **xpathCheck** این تابع که در پایین نیز آورده شده است ابتدا سیگنال دارای اشکال را به یک لیست اضافه می کند، سپس بر روی تمام سیگنال های ما حرکت کرده، اگر ورودی یک گیت در لیست مدنظر وجود داشته باشد و خروجی آن نامشخص باشد، خروجی آن را به لیست اضافه می کند. در نهایت چک می کنیم که حداقل یکی از خروجی های اصلی مدار در لیست مطرح شده وجود داشته باشد. درواقع در این تابع چک می کنیم آیا سیگنال دچار اشکال میتواند در خروجی اثر داشته باشد یا خیر.

```

def xpathCheck(self, netList, fault) -> bool:
    faultyNet = netList[fault[0]]

    faultyNets = [faultyNet]

    for i in netList.keys():
        net = netList[i]

        if net[2] == "from":
            if net[3] in [fnet[1] for fnet in faultyNets]:
                faultyNets.append(net)

        elif net[2] == "inpt":
            continue

        else:
            startInputsIndex = len(net) - 1 - int(net[4])
            endInputsIndex = len(net) - 1
            if any([(fnet[0] in net[startInputsIndex:endInputsIndex]) for fnet in faultyNets]):
                inputNets = [netList[j] for j in net[startInputsIndex: endInputsIndex]]
                inputValues = [inputNet[len(inputNet) - 1] for inputNet in inputNets]
                if self.getValueIn5(net[2],inputValues) in (ValueType.Unknown, ValueType.One_Zero, ValueType.Zero_One):
                    faultyNets.append(net)

    if faultyNets[-1][2] != "from" and faultyNets[-1][3] == "0":
        return True

    return False

```

تابع **obtainObjective** در این تابع قرار است objective الگوریتم PODEM به عنوان خروجی بازگردانده شود. ابتدا یکی از گیت‌های d-frontier که به خروجی نزدیکتر است انتخاب می‌شود، سپس به ورودی‌های آن با توجه به مقدار controllability آن‌ها مقداری تخصیص می‌یابد و به عنوان objective بازگردانده می‌شود. ورودی‌ای که کمترین میزان controllability را دارد به شرطی که خروجی را از حالت نامشخص خارج نکند مقدار کنترل‌کننده نباشد انتخاب می‌شود.

```
def obtainObjective(self, netList, fault, controllabilities : dict) -> list:

    if faultyNet[-1] == ValueType.Unknown:
        return [fault[0], ValueType.One if fault[1] == FaultType.SA0 else ValueType.Zero]

    else:
        keys = list(netList.keys())
        for i in range(len(keys) - 1, -1, -1):
            net = netList[keys[i]]

            if net[-1] != ValueType.Unknown:
                continue

            if net[2] == "from" or net[2] == "inpt":
                continue

            else:
                startInputsIndex = len(net) - 1 - int(net[4])
                endInputsIndex = len(net) - 1
                inputNets = [netList[j] for j in net[startInputsIndex: endInputsIndex]]
                inputValues = [inputNet[-1] for inputNet in inputNets]

                # check is dfrontier
                isDfrontier = False
                for inputValue in inputValues:
                    if inputValue == ValueType.One_Zero or inputValue == ValueType.Zero_One:
                        isDfrontier = True
                        break
                if not isDfrontier:
                    continue
```

```

minControlNetIndex = [None, None]
minControlValue = [math.inf, math.inf]

j = 0
for inputNet in inputNets:
    if inputNet[-1] == ValueType.Unknown:
        netControl = controllabilities[inputNet[0]]["CC"]
        if netControl[0] < minControlValue[0]:
            minControlValue[0] = netControl[0]
            minControlNetIndex[0] = j
        if netControl[1] < minControlValue[1]:
            minControlValue[1] = netControl[1]
            minControlNetIndex[1] = j
    j += 1

if minControlValue[0] < minControlValue[1]:
    inputValues[minControlNetIndex[0]] = ValueType.Zero
    if self.getValueIn5(net[2], inputValues) in (ValueType.Unknown, ValueType.Zero_One, ValueType.One_Zero):
        return [inputNets[minControlNetIndex[0]][0], ValueType.Zero]
    else:
        inputValues[minControlNetIndex[0]] = ValueType.Unknown
        inputValues[minControlNetIndex[1]] = ValueType.One
        if self.getValueIn5(net[2], inputValues) in (ValueType.Unknown, ValueType.Zero_One, ValueType.One_Zero):
            return [inputNets[minControlNetIndex[1]][0], ValueType.One]

```

```

else:
    inputValues[minControlNetIndex[1]] = ValueType.One
    if self.getValueIn5(net[2], inputValues) in (ValueType.Unknown, ValueType.Zero_One, ValueType.One_Zero):
        return [inputNets[minControlNetIndex[1]][0], ValueType.One]
    else:
        inputValues[minControlNetIndex[1]] = ValueType.Unknown
        inputValues[minControlNetIndex[0]] = ValueType.Zero
        if self.getValueIn5(net[2], inputValues) in (ValueType.Unknown, ValueType.Zero_One, ValueType.One_Zero):
            return [inputNets[minControlNetIndex[0]][0], ValueType.Zero]

```

تابع backtrace این تابع objective را دریافت کرده یکی از ورودی‌های مدار را مقداردهی می‌کند. در هر بار گذر از یک گیت اگر برای رسیدن به objective نیاز به مقداردهی به تمام ورودی‌های این گیت باشد، ورودی‌ای که controllability آن بیشتر است را انتخاب کرده و مقداردهی می‌کند، در غیر این صورت ورودی‌ای از آن گیت که controllability آن کمتر است انتخاب می‌شود. این عمل را تا جایی که به یکی از ورودی‌های اصلی مدار برسد انجام می‌دهد.

```

def backTrace(self, netList, objective: list, controllabilies : dict) -> list:
    objectiveNet = netList[objective[0]]
    objectiveValue = objective[1]

    while True:
        if objectiveNet[2] == "inpt":
            return [objectiveNet[0], objectiveValue]

        elif objectiveNet[2] == "from":
            for i in netList.keys():
                net = netList[i]
                if net[1] == objectiveNet[3]:
                    objectiveNet = net
                    break

        else:
            startInputsIndex = len(objectiveNet) - 1 - int(objectiveNet[4])
            endInputsIndex = len(objectiveNet) - 1
            inputNets = [netList[j] for j in objectiveNet[startInputsIndex: endInputsIndex]]
            inputValues = [inputNet[-1] for inputNet in inputNets]

            (variable) minControlValue: list[int]
            minControlValue = [1000_000_000, 1000_000_000]
            maxControlNetIndex = [None, None]
            maxControlValue = [0, 0]

```

```

j = 0
for inputNet in inputNets:
    if inputNet[-1] == ValueType.Unknown:
        netControl = controllabilities[inputNet[0]]["CC"]
        if netControl[0] < minControlValue[0]:
            minControlValue[0] = netControl[0]
            minControlNetIndex[0] = j
        if netControl[1] < minControlValue[1]:
            minControlValue[1] = netControl[1]
            minControlNetIndex[1] = j
        if netControl[0] > maxControlValue[0]:
            maxControlValue[0] = netControl[0]
            maxControlNetIndex[0] = j
        if netControl[1] > maxControlValue[1]:
            maxControlValue[1] = netControl[1]
            maxControlNetIndex[1] = j
    j += 1

assignObjective = False

if minControlValue[0] < minControlValue[1]:
    inputValues[minControlNetIndex[0]] = ValueType.Zero
    if self.getValueIn5(objectiveNet[2], inputValues) == objectiveValue:
        objectiveNet = inputNets[minControlNetIndex[0]]
        objectiveValue = ValueType.Zero
        assignObjective = True
    inputValues[minControlNetIndex[0]] = ValueType.Unknown

```

```

if not assignObjective:
    inputValues[minControlNetIndex[0]] = ValueType.Zero
    if self.getValueIn5(objectiveNet[2], inputValues) == objectiveValue:
        objectiveNet = inputNets[minControlNetIndex[0]]
        objectiveValue = ValueType.Zero
        assignObjective = True
    inputValues[minControlNetIndex[0]] = ValueType.Unknown

if not assignObjective:
    if maxControlValue[0] > maxControlValue[1]:
        inputValues[maxControlNetIndex[0]] = ValueType.Zero
        if self.getValueIn5(objectiveNet[2], inputValues) in (ValueType.Unknown, ValueType.Zero_One, ValueType.One_Zero):
            objectiveNet = inputNets[maxControlNetIndex[0]]
            objectiveValue = ValueType.Zero
            assignObjective = True
        inputValues[maxControlNetIndex[0]] = ValueType.Unknown

if not assignObjective:
    inputValues[maxControlNetIndex[1]] = ValueType.One
    if self.getValueIn5(objectiveNet[2], inputValues) in (ValueType.Unknown, ValueType.Zero_One, ValueType.One_Zero):
        objectiveNet = inputNets[maxControlNetIndex[1]]
        objectiveValue = ValueType.One
        assignObjective = True
    inputValues[maxControlNetIndex[1]] = ValueType.Unknown

```

```

if not assignObjective:
    inputValues[maxControlNetIndex[0]] = ValueType.Zero
    if self.getValueIn5(objectiveNet[2], inputValues) in (ValueType.Unknown, ValueType.Zero_One, ValueType.One_Zero):
        objectiveNet = inputNets[maxControlNetIndex[0]]
        objectiveValue = ValueType.Zero
        assignObjective = True
    inputValues[maxControlNetIndex[0]] = ValueType.Unknown

```

تابع `getValueIn5` این تابع مقدار خروجی یک گیت را بر اساس مقادیر ورودی آن مشخص میکند، با این تفاوت که مقادیر ما منطق 5 مقداری (U, 0, 1, 0/1, 1/0) هستند. شیوه عملکرد ساده ای دارد، یک بار خروجی را در حالت سالم و بار دیگر در حالت اشکال مطابق با آنچه در فاز 1 پروژه بود بدست می آورد، از مقایسه دو مقدار به دست آمده خروجی را تعیین می کند.

```

def getValueIn5(self, net, inputValues : list):
    inputValuesCopy = inputValues.copy()
    for i in range(len(inputValuesCopy)):
        if inputValuesCopy[i] == ValueType.Zero_One:
            inputValuesCopy[i] = ValueType.Zero
        elif inputValuesCopy[i] == ValueType.One_Zero:
            inputValuesCopy[i] = ValueType.One

    correctResult = self.testabilityObj.getValue(net, inputValuesCopy)

    inputValuesCopy = inputValues.copy()
    for i in range(len(inputValuesCopy)):
        if inputValuesCopy[i] == ValueType.Zero_One:
            inputValuesCopy[i] = ValueType.One
        elif inputValuesCopy[i] == ValueType.One_Zero:
            inputValuesCopy[i] = ValueType.Zero

    faultyResult = self.testabilityObj.getValue(net, inputValuesCopy)

    if correctResult == faultyResult:
        return correctResult
    elif correctResult == ValueType.Zero and faultyResult == ValueType.One:
        return ValueType.Zero_One
    elif correctResult == ValueType.One and faultyResult == ValueType.Zero:
        return ValueType.One_Zero
    else:
        return ValueType.Unknown

```

تابع **imply** این تابع عمل Implication را انجام داده و مقادیر تمام سیگنال‌ها را با توجه به مقدار سیگنال‌های ورودی و محل اشکال مشخص می‌کند. عملکرد این تابع مطابق فاز 1 است منتهی بجای منطق 4 مقداری از منطق 5 مقداری و تابع `getValueIn5` استفاده میکند.

```
def imply(self, netList, testStack, fault):

    self.clearNetList(netList)

    for tnet in testStack:
        netList[tnet[0]][-1] = tnet[1]

    for i in netList.keys():
        net = netList[i]
        if net[2] == 'from':
            for j in netList.keys():
                if netList[j][1] == net[3]:
                    inputNet = netList[j]
                    net[len(net) - 1] = inputNet[len(inputNet) - 1]
                    break
            elif net[2] != "inpt":
                startInputsIndex = len(net) - 1 - int(net[4])
                endInputsIndex = len(net) - 1
                inputNets = [netList[j] for j in net[startInputsIndex: endInputsIndex]]
                inputValues = [inputNet[len(inputNet) - 1] for inputNet in inputNets]
                net[len(net) - 1] = self.getValueIn5(net[2], inputValues)

        if net[0] == fault[0]:
            if fault[1] == FaultType.SA0:
                if net[-1] == ValueType.One:
                    net[-1] = ValueType.One_Zero
            else:
                if net[-1] == ValueType.Zero:
```

تابع **checkDetected** این تابع بررسی می کند آیا اثر اشکال در خروجی ظاهر شده است یا خیر.

```
def checkDetected(self, netList) -> bool:
    for i in netList.keys():
        net = netList[i]
        if net[2] == 'from' or net[2] == "inpt":
            continue
        elif net[3] == "0":
            # outputs
            if net[-1] == ValueType.One_Zero or net[-1] == ValueType.Zero_One:
                return True

    return False
```

تابع **backTrack** ما مقادیر اختصاص یافته به ورودی ها را در یک پشته ذخیره می کنیم. این تابع با دریافت این پشته، آخرین ورودی ای که مقدار دیگر آن را تخصیص نداده ایم را انتخاب و مقدار دیگر آن را تخصیص می دهد و پشته ما را بروز می کند. اگر تخصیص همه حالات را بررسی کرده باشیم خروجی False تولید می کند.

```
def backTrack(self, testStack : list) -> bool:
    if False in [testNet[2] for testNet in testStack]:
        i = -1
        while testStack[i][2]:
            i -= 1
        tNet = testStack[i]
        for j in range(-1, i, -1):
            testStack[j][2] = False
        if tNet[1] == ValueType.One:
            tNet[1] = ValueType.Zero
        else:
            tNet[1] = ValueType.One
        tNet[2] = True
        return True
    return False
```

تابع PODEM این تابع به ازای یک اشکال مشخص الگوریتم PODEM را اجرا می کند اگر الگوریتم بتواند بردار تست تولید کند، آن را برمی گرداند.

```
4 def PODEM(self, netList, fault) -> dict:
5     state = "Start"
6     objective = None
7     testStack = []
8     controllabilies = self.testabilityObj.CC_CO
9
10    while True:
11
12        if state == "Start":
13            print("start")
14            # assign controllabilies of all signals
15            self.clearNetList(netList)
16            # print(netList)
17            state = "XPathCheck"
18
19        elif state == "XPathCheck":
20            print("xpath")
21            if self.xpathCheck(netList, fault):
22                state = "Objective"
23            else:
24                state = "BackTrack"
25
26        elif state == "Objective":
27            print("objective")
28            objective = self.obtainObjective(netList, fault, controllabilies)
29            if objective != None:
30                state = "BackTrace"
```

```

else:
    state = "Finish_Untestable"

elif state == "BackTrace":
    print("backtrace")
    (pi, v) = self.backTrace(netList, objective, controllabilities)
    testStack.append([pi, v, False])
    state = "Implication"

elif state == "Implication":
    print("implication")
    self.imply(netList, testStack, fault)
    state = "CheckDetected"

elif state == "CheckDetected":
    print("checkdetected")
    if self.checkDetected(netList):
        state = "Finish_Success"
    else:
        state = "XPathCheck"

elif state == "BackTrack":
    print("backtrack")
    if self.backTrack(testStack):
        state = "Implication"
    else:
        state = "Finish_Untestable"

elif state == "Finish_Success":

elif state == "Finish_Success":
    print("finish")
    testVector = self.generateTestVector(testStack)
    return testVector

else:
    return {}

```

تابع globalPODEM این تابع تمام اشکالات را گرفته و برای هر یک از آنها تابع PODEM را فراخوانی می کند و در نهایت لیست تمام بردارهای تست را باز می گرداند.

```
def globalPODEM (self, netList, faultList) -> list[dict]:
    testVectors = []

    for fault in faultList:
        testVector = self.PODEM(netList, fault)
        testVectors.append(testVector)

    return testVectors
```

تابع **exportTestVectors** با استفاده از این تابع، فایل اشکال و فایل Fault خوانده می‌شود، تابع globalPODEM بر روی مدار اعمال شده و بردارهای تست نتیجه شده را بر روی فایل ذخیره می‌کند.

```
def exportTestVectors(inputFilePath : str, faultFilePath : str, testVectorsPath : str, SCOAPpath: str):
    with open(inputFilePath, 'r') as inputFile:
        with open(faultFilePath, 'r') as faultFile:
            faultListToprint = []
            for line in faultFile.readlines():
                splittedLine = line.strip().split()
                splittedLine[1] = FaultType.SA0 if splittedLine[1] == "sa0" else FaultType.SA1
                faultListToprint.append(splittedLine)
            print(faultListToprint)
            netList = obj.readISC(inputFile)
            faultList = obj.readFaults(faultFile, netList)
            obj.CC_CO = {}
            obj.calculateControllability(netList)
            obj.calculateObservability(netList)
            testVectors = podemObj.globalPODEM(netList, faultList)
            print(faultList)
            inputs = []
            for i in netList.keys():
                net = netList[i]
                if net[2] == "inpt":
                    inputs.append(net[0])
            with open(testVectorsPath, "w") as testF:
                testF.write(f"{'net':<8}")
                testF.write(f"{'fault':<8}")
                for input in inputs:
                    testF.write(f"{'input':<4}")
```

```

testF.write("\n-----" + len(inputs) * "----")
f = 0
for testVector in testVectors:
    testF.write("\n")
    testF.write(f"{faultListToprint[f][0]:<8}")
    testF.write(f"{faultListToprint[f][1].value:<8}")
    if len(testVector.keys()) == 0:
        testF.write("none found")
    else:
        for input in inputs:
            if input in testVector:
                testF.write(f"{testVector[input].value:<4}")
            else:
                testF.write(f"{\"U\":<4}")
        f += 1
with open(SCOAPpath, "w") as SCOAPF:
    SCOAPF.write(f"{\"net\":<8}")
    SCOAPF.write(f"{\"CC0\":<6}")
    SCOAPF.write(f"{\"CC1\":<6}")
    SCOAPF.write(f"{\"CO\":<6}")
    SCOAPF.write(f"\n-----\n")
    CC_CO = obj.CC_CO
    for net in CC_CO:
        SCOAPF.write(f"{net:<8}")
        SCOAPF.write(f"{CC_CO[net][\"CC\"][0]:<6}")
        SCOAPF.write(f"{CC_CO[net][\"CC\"][1]:<6}")
        SCOAPF.write(f"{CC_CO[net][\"CO\"]:<6}\n")

```

تابع **runExportVectors** اسم مدار را میگیرد و Exporttestvectors را برای آن اجرا میکند.

```

4 def runExportVectors(circuitName: str):
5     exportTestVectors(f"./Inputs/{circuitName}.isc", f"./Inputs/{circuitName}_fault.txt", f"./Outputs/{circuitName}_test_vectors.txt")
6
7
8 runExportVectors("c5")
9 runExportVectors("c1")
0 runExportVectors("c17")
1 runExportVectors("c432")
2 runExportVectors(["c6288"])

```

آزمایش

با فراخوانی تابع **runExportVectors** برای چندتا مدار مختلف آزمایش خود را انجام دادیم. این مدار را به ازای تمام اشکالهای مختلف با استفاده از فایل زیر آزمایش کرده ایم. برای مدار C1 مقادیر اشکال به این صورت است.

1	1	sa0
2	1	sa1
3	2	sa0
4	2	sa1
5	3	sa0
6	3	sa1

خروجی زیر، خروجی پیاده سازی ما می باشد.

1	net	fault	1	2
2	-----			
3	1	sa-0	1	1
4	1	sa-1	0	1
5	2	sa-0	1	1
6	2	sa-1	1	0
7	3	sa-0	0	U
8	3	sa-1	1	1

مدار C5 فایل Fault.

فایل اشکالها (تمام اشکالهای ممکن):

1	1	sa0
2	1	sa1
3	2	sa0
4	2	sa1
5	4	sa0
6	4	sa1
7	3	sa0
8	3	sa1
9	5	sa0
10	5	sa1
11		

خروجی (بردارهای تست):

1	net	fault	1	2	4
2	-----				
3	1	sa-0	1	1	1
4	1	sa-1	0	1	1
5	2	sa-0	1	1	1
6	2	sa-1	1	0	1
7	4	sa-0	0	U	1
8	4	sa-1	0	U	0
9	3	sa-0	0	U	1
10	3	sa-1	1	1	1
11	5	sa-0	0	U	1
12	5	sa-1	U	U	0

مدار C17 فایل Fault.

فایل اشکال ها (تمام اشکالهای ممکن):

1	1	sa0
2	1	sa1
3	2	sa0
4	2	sa1
5	3	sa0
6	3	sa1
7	6	sa0
8	6	sa1
9	7	sa0
10	7	sa1
11	10	sa0
12	10	sa1
13	3_1	sa0
14	3_1	sa1
15	3_2	sa0
16	3_2	sa1
17	11	sa0
18	11	sa1
19	16	sa0
20	16	sa1
21	11_1	sa0
22	11_1	sa1
23	11_2	sa0
24	11_2	sa1
25	19	sa0
26	19	sa1
27	22	sa0
28	22	sa1
29	16_1	sa0
30	16_1	sa1
31	16_2	sa0
32	16_2	sa1
33	23	sa0
34	23	sa1

خروجی (بردارهای تست):

1	net	fault	1	2	3	6	7
2	-----						
3	1	sa-0	1	0	1	U	U
4	1	sa-1	0	0	1	U	U
5	2	sa-0	U	1	0	U	U
6	2	sa-1	U	0	0	U	U
7	3	sa-0	U	0	1	1	1
8	3	sa-1	U	0	0	1	1
9	6	sa-0	U	0	1	1	1
10	6	sa-1	U	0	1	0	1
11	7	sa-0	U	0	0	U	1
12	7	sa-1	U	0	0	U	0
13	10	sa-0	0	0	U	U	U
14	10	sa-1	1	0	1	U	U
15	3_1	sa-0	1	0	1	U	U
16	3_1	sa-1	1	0	0	U	U
17	3_2	sa-0	U	0	1	1	1
18	3_2	sa-1	U	0	0	1	1

19	11	sa-0	U	0	0	U	1
20	11	sa-1	U	0	1	1	1
21	16	sa-0	U	0	U	U	0
22	16	sa-1	U	1	0	U	U
23	11_1	sa-0	U	1	0	U	U
24	11_1	sa-1	U	1	1	1	U
25	11_2	sa-0	U	0	0	U	1
26	11_2	sa-1	U	U	1	1	1
27	19	sa-0	U	0	U	U	0
28	19	sa-1	U	0	0	U	1
29	22	sa-0	1	U	1	U	U
30	22	sa-1	0	0	U	U	U
31	16_1	sa-0	0	0	U	U	U
32	16_1	sa-1	U	1	0	U	U
33	16_2	sa-0	U	0	U	U	0
34	16_2	sa-1	U	1	0	U	0
35	23	sa-0	U	1	0	U	U
36	23	sa-1	U	0	U	U	0

تعدادی از اشکال ها برای نمونه به مدار C6288 به شکل زیر هستند.

1	4925	sa0
2	3541	sa1
3	1124	sa0
4	4256	sa1

خروجی (بردارهای تست):

net	fault	1	18	35	52	69	86	103	120	137	154	171	188	205	222	239	256	273	290	307	324	341	358	375	392	409	426	443
4925	sa-0	U	U	U	U	U	1	1	1	1	1	1	1	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
3541	sa-1	U	U	U	U	1	1	1	1	1	1	1	1	1	0	1	0	0	0	0	1	1	1	1	1	1	1	1
1124	sa-0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4256	sa-1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	U	1	1	1	1	1	1	1	1	1	1	1

	324	341	358	375	392	409	426	443	460	477	494	511	528
1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	U